

Univerzita Jana Evangelisty Purkyně
v Ústí nad Labem
Přírodovědecká fakulta



GoodAccess CLI klient pro Linux

BAKALÁŘSKÁ PRÁCE

Vypracoval: Valdemar Pospíšil

Vedoucí práce: Ing. Jiří Hromádka

Studijní program: Aplikovaná informatika

Studijní obor:

ÚSTÍ NAD LABEM 2026

Namísto žlutých stránek vložte digitálně podepsané zadání kvalifikační práce poskytnuté vedoucím katedry.

Zadání musí zaujímat právě dvě strany.

Zadání je nutno vložit jako PDF pomocí některého nástroje, který umožňuje editaci dokumentů (se zachováním elektronického podpisu).

V Linuxe lze například použít příkaz `pdftk`.

Prohlášení

Prohlašuji, že jsem tuto bakalářskou práci vypracoval samostatně a použil jen pramenů, které cituji a uvádím v přiloženém seznamu literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., ve znění zákona č. 81/2005 Sb., autorský zákon, zejména se skutečností, že Univerzita Jana Evangelisty Purkyně v Ústí nad Labem má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Jana Evangelisty Purkyně v Ústí nad Labem oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladu, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

V Ústí nad Labem dne 13. února 2026

Podpis:

Děkuji vedoucímu práce **Ing. Jiřímu Hromádkovi**
za neocenitelné rady a pomoc při tvorbě bakalářské práce.

Abstrakt:

V současné době nabízí GoodAccess VPN řešení především prostřednictvím grafického uživatelského rozhraní, což nevyhovuje všem uživatelským scénářům. Zejména v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů preferují administrátoři a pokročilí uživatelé řešení příkazové řádky.

Hlavním cílem této práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích s využitím WireGuard protokolu. Práce se zabývá návrhem architektury, bezpečným ukládáním dat a implementací komunikace se systémovou službou pomocí IPC.

Klíčová slova: VPN, GoodAccess, CLI, Linux, Go, .NET, WireGuard, IPC, Clean Architecture

GOODACCESS CLI CLIENT FOR LINUX

Abstract:

Currently, GoodAccess VPN offers solutions primarily through a graphical user interface, which does not suit all user scenarios. Especially in server environments, automated systems, and DevOps workflows, administrators and advanced users prefer command-line solutions.

The main goal of this work is to design and implement a functional prototype of a CLI application for managing GoodAccess VPN connections on Linux distributions using the WireGuard protocol. The work deals with the design of the architecture, secure data storage, and implementation of communication with the system service using IPC.

Keywords: VPN, GoodAccess, CLI, Linux, Go, .NET, WireGuard, IPC, Clean Architecture

Obsah

Úvod	13
1. Teoretická východiska	15
1.1. Rozhraní příkazové řádky	15
1.2. Architektura VPN a principy Zero Trust	16
1.3. Architektura operačního systému Linux	18
1.4. Implementační technologie	21
1.5. Distribuce softwaru	22
1.6. Metodika vývoje	22
2. Analýza a specifikace požadavků	23
2.1. Identifikace zúčastněných stran	23
2.2. Funkční požadavky	23
2.3. Nefunkční požadavky	24
2.4. Analýza architektury	24
3. Návrh řešení	27
3.1. Celková architektura systému	27
3.2. Návrh komunikace (IPC)	27
3.3. Návrh uživatelského rozhraní (CLI)	27
3.4. Bezpečnostní model	27
3.5. Datový model a konfigurace	27
4. Implementace	29
4.1. Struktura projektu a nástroje	29
4.2. Implementace klienta (Frontend)	29
4.3. Úpravy backendové služby	29
4.4. Integrace WireGuardu	29
4.5. Distribuční balíčky a Systemd	29
5. Testování a validace	31
5.1. Metodika testování	31
5.2. Jednotkové testy (Unit Testing)	31
5.3. Integrační a systémové testy	31
5.4. Testování na různých distribucích	31

Závěr	33
A. Externí přílohy	37
B. Další přílohy	39

Úvod

V posledních letech došlo k zásadnímu posunu v organizaci práce a správě síťové infrastruktury. S masivním přechodem na práci na dálku a distribuované týmy se tradiční modely zabezpečení sítě ukazují jako nedostatečné. Organizace stále častěji adoptují architektury typu Zero Trust Network Access (ZTNA), kde je bezpečné VPN připojení klíčovým prvkem nejen pro koncové uživatele, ale i pro serverovou infrastrukturu.

Platforma GoodAccess v současné době nabízí řešení primárně prostřednictvím grafického uživatelského rozhraní (GUI). Toto řešení je vhodné pro běžné uživatele na osobních počítačích, avšak naráží na limity v prostředí serverů, automatizovaných systémů a DevOps pracovních postupů.

Motivace

V serverovém prostředí Linux, které je dominantní platformou pro cloudovou infrastrukturu, často není grafické rozhraní dostupné (tzv. headless servery). Administrátoři a DevOps inženýři vyžadují nástroje, které lze ovládat z příkazové řádky, snadno skriptovat a integrovat do CI/CD pipelin. Absence nativního CLI (Command Line Interface) klienta pro Linux představuje pro GoodAccess značné omezení v možnosti nasazení v těchto scénářích.

Cíle práce

Hlavním cílem této bakalářské práce je navrhnout a implementovat funkční prototyp CLI aplikace pro správu GoodAccess VPN připojení na Linux distribucích. Důraz je kladen na vytvoření "lehkého" a nativního uživatelského rozhraní, které efektivně komunikuje se systémovou službou a využívá stávající podnikovou logiku pro správu VPN tunelů. Práce má ambici ukázat kompletní inženýrský proces vývoje softwaru od analýzy požadavků až po distribuci finálních balíčků.

Dílčí cíle práce zahrnují:

- Analýzu a návrh architektury založené na principu oddělení logiky od prezentace (tzv. "stupid frontend" přístup).
- Implementaci meziprocesní komunikace (IPC) mezi uživatelským rozhraním v jazyce Go a systémovým modulem v .NET prostřednictvím Unix Domain Sockets.

- Vytvoření modulu v .NET pro bezpečné řízení VPN tunelů (OpenVPN, WireGuard) s využitím existujících komponent platformy GoodAccess.
- Návrh a realizaci automatizovaného testování a balíčkování aplikace pro distribuce Debian a Fedora.

Struktura práce

Práce je rozdělena do několika logických celků. Kapitola 1 se věnuje teoretickým východiskům, popisuje architekturu Linuxu, použité VPN protokoly a metody meziprocesové komunikace. Kapitola 2 definuje požadavky na systém a analyzuje možné architektonické přístupy. Následující kapitoly se poté věnují samotnému návrhu, implementaci a testování výsledného řešení.

1. Teoretická východiska

Tato kapitola analyzuje klíčové technologie, architektonické principy a bezpečnostní standardy nezbytné pro návrh moderního CLI nástroje v prostředí OS Linux. Text postupuje od obecné problematiky příkazové řádky přes síťové protokoly až po specifika implementačních platforem .NET a Go.

1.1. Rozhraní příkazové řádky

Rozhraní příkazové řádky (CLI – Command Line Interface) představuje jeden z nejstarších, avšak stále nejvýkonnějších způsobů interakce uživatele s počítačem. Navzdory masivnímu rozšíření grafických uživatelských rozhraní (GUI) v 90. letech 20. století zůstává CLI nepostradatelným nástrojem pro systémové administrátory, vývojáře a pokročilé uživatele, zejména v prostředí operačních systémů typu Unix a Linux.

Role CLI v moderní správě systémů

Hlavním důvodem preference příkazové řádky v profesionálním prostředí je efektivita, rychlost a snadná automatizace. Zatímco grafické rozhraní je navrženo pro intuitivní objevování funkcí pomocí vizuálních prvků, CLI se zaměřuje na precizní a opakovatelné provádění úkolů. Administrátor může pomocí jediného příkazu nebo krátkého skriptu provést operaci na stovkách serverů současně, což je v rámci GUI prakticky neproveditelné.

Dalším kritickým faktorem je vzdálená správa. Protokol SSH (Secure Shell), který je de facto standardem pro správu linuxových serverů, je primárně textově orientovaný. Přenos textových příkazů a jejich výstupů vyžaduje minimální síťovou propustnost, což umožňuje spolehlivou správu systémů i přes nestabilní nebo pomalé připojení. V prostředí cloudu a kontejnerizace (např. Docker, Kubernetes), kde systémy často nemají žádné grafické výstupní zařízení (headless režim), je CLI jedinou možností interakce.

Principy návrhu a standardy CLI aplikací

Kvalitní CLI aplikace se řídí souborem ustálených principů, které zajišťují jejich předvídatelnost a vzájemnou spolupráci. Eric S. Raymond ve své knize *The Art of Unix Programming*

definuje tzv. „Unixovou filosofii“, jejímž základem je tvorba malých, jednoúčelových nástrojů, které spolu efektivně komunikují [1]. Tento přístup umožňuje skládat komplexní operace z jednoduchých komponent pomocí mechanismu `rour` (pipes).

Základním stavebním kamenem interakce v CLI jsou standardní datové proudy:

- **Standardní vstup (`stdin`):** Zdroj dat pro aplikaci, typicky klávesnice nebo výstup jiného programu.
- **Standardní výstup (`stdout`):** Primární kanál pro výstup dat, typicky terminál.
- **Standardní chybový výstup (`stderr`):** Oddělený kanál pro chybová hlášení a diagnostiku, což umožňuje uživateli přesměrovat data do souboru, zatímco chyby stále vidí na obrazovce.

Důležitou roli hraje také syntaxe příkazů. Moderní nástroje se snaží dodržovat standardy jako POSIX nebo GNU konvence pro argumenty a přepínače (flags). Krátké přepínače (např. `-v`) jsou určeny pro rychlé psaní, zatímco dlouhé varianty (např. `--verbose`) zvyšují čitelnost skriptů a dokumentace.

CLI versus TUI

V rámci terminálu se můžeme setkat také s textovým uživatelským rozhraním (TUI – Text User Interface). Na rozdíl od klasického CLI, které vypisuje řádky textu v sekvenčním pořadí, TUI využívá celou plochu terminálu k vykreslování vizuálních prvků, jako jsou okna, menu, tabulky nebo interaktivní grafy, často s využitím knihoven jako `ncurses` nebo moderních frameworků v jazyce Go (např. `Bubble Tea`).

Hlavní rozdíl spočívá v komplexitě a způsobu interakce. CLI je ideální pro jednorázové příkazy a automatizaci v nekonečném proudu (batch processing). TUI je naproti tomu vhodnější pro interaktivní nástroje, kde uživatel potřebuje neustálý přehled o stavu systému nebo kde je navigace v datech pomocí klávesových zkratk efektivnější než vypisování parametrů (např. textové editory, správci souborů nebo dashboardy pro monitoring).

1.2. Architektura VPN a principy Zero Trust

Tradiční pojetí zabezpečení sítě, založené na ochraně perimetru, prochází v posledních letech zásadní transformací. Tato sekce popisuje evoluci od klasických VPN tunelů k moderní architektuře Zero Trust Network Access (ZTNA).

Tradiční VPN a perimetrová bezpečnost

Virtuální privátní síť (VPN) je technologie, která umožňuje vytvořit bezpečné, šifrované spojení (tunel) mezi dvěma body v rámci nezabezpečené sítě, jako je internet. Princip tu-

nelování spočívá v zapouzdření (enkapsulaci) síťových paketů do jiného protokolu, který zajišťuje integritu a důvěrnost přenášených dat.

Historicky se organizace spoléhaly na model „hrad a příkop“ (Castle-and-Moat). V tomto modelu je vnitřní síť považována za důvěryhodnou zónu, která je od vnějšího světa oddělena firewallem (příkopem). VPN v tomto kontextu slouží jako „padací most“, který umožňuje vzdáleným uživatelům vstoupit do vnitřní sítě. Jakmile však uživatel získá přístup, má často široký dosah k mnoha zdrojům v rámci sítě. Tento přístup trpí kritickou slabinou: pokud útočník kompromituje jediné zařízení uvnitř perimetru, může se v síti volně pohybovat (laterální pohyb) a napadat další systémy, protože vnitřní síť mu implicitně důvěřuje.

Zero Trust Network Access (ZTNA) a GoodAccess

Filozofie Zero Trust (nulová důvěra) tento model opouští a vychází z principu „nikdy nedůvěřuj, vždy prověřuj“ (Never trust, always verify). V architektuře ZTNA není důvěra nikdy udělena na základě fyzického umístění uživatele nebo jeho IP adresy. Každý pokus o přístup k jakémukoliv zdroji musí být individuálně autentizován, autorizován a šifrován.

Klíčovým prvkem ZTNA je Software Defined Perimeter (SDP). SDP vytváří dynamickou, virtuální hranici kolem aplikací a služeb, čímž je činí „neviditelnými“ pro veřejný internet i pro neautorizované uživatele v lokální síti. GoodAccess implementuje tyto principy formou cloudové platformy, která nahrazuje tradiční VPN brány [2]. Místo statických IP adres využívá identitu uživatele (často propojenou s SSO/OIDC) a kontext zařízení (tzv. Device Posture) k dynamickému přidělování přístupových práv k jednotlivým systémům [3]. Tím se radikálně snižuje plocha možného útoku, protože uživatel vidí pouze ty zdroje, ke kterým má explicitní oprávnění.

Srovnání protokolů WireGuard a OpenVPN

Naproti tomu WireGuard je moderní protokol navržený s důrazem na jednoduchost, rychlost a moderní kryptografii [4]. S rozsahem přibližně 4 000 řádků kódu je WireGuard snadno auditovatelný a díky implementaci přímo v jádře Linuxu dosahuje výrazně vyšší propustnosti a nižší latence než OpenVPN. Z pohledu správy systému se WireGuard chová jako standardní síťové rozhraní, což zjednodušuje konfiguraci směrování a integraci do systémových nástrojů. Pro dynamické enterprise prostředí, které GoodAccess obsluhuje, je WireGuard ideální volbou díky rychlému navazování spojení (handshake) a vysoké odolnosti vůči změnám síťového prostředí.

1.3. Architektura operačního systému Linux

Tato sekce analyzuje klíčové mechanismy operačního systému Linux, které jsou nezbytné pro běh systémových služeb a správu síťových rozhraní.

Struktura systému: User Space a Kernel Space

Linux dělí paměťový prostor na dvě privilegovaně odlišné části. Toto rozdělení je kritické pro stabilitu a bezpečnost systému.

- **Kernel Space (Prostor jádra):** Zde běží jádro operačního systému, ovladače zařízení a kritické síťové moduly, včetně implementace protokolu WireGuard. Kód v tomto prostoru má plný přístup k hardwaru.
- **User Space (Uživatelský prostor):** Zde běží běžné aplikace, včetně námi vyvíjeného CLI klienta a backendové služby.

Toto oddělení, často označované jako *privilege separation*, zajišťuje, že chybný nebo škodlivý kód v uživatelské aplikaci nemůže přímo ohrozit integritu celého systému. Brian Ward ve své knize *How Linux Works* zdůrazňuje, že zatímco uživatelský prostor je místem, kde probíhá většina „reálné akce“ aplikací, jádro slouží jako arbitr a správce všech hardwarových prostředků [5].

Aplikace v uživatelském prostoru nemohou přímo přistupovat k hardwaru. Musí využívat tzv. *systémová volání* (syscalls) pro komunikaci s jádrem. Přejít mezi těmito prostory vyžaduje tzv. *přepnutí kontextu* (context switch), což je operace s nenulovou režii. V kontextu VPN řešení je toto kritické při zpracování síťového provozu – například standardní OpenVPN běží v uživatelském prostoru a musí každý paket kopírovat mezi prostory přes virtuální síťové rozhraní (TUN/TAP), což může omezovat propustnost při vysokém zatížení. Naopak WireGuard, díky své implementaci přímo v prostoru jádra, tuto režii minimalizuje a umožňuje efektivnější zpracování datových toků [4].

Správa procesů a životní cyklus služeb

Proces je v Linuxu definován jako instance běžícího programu v paměti. Každý proces má svůj unikátní identifikátor (PID) a je spravován jádrem, které mu přiděluje procesorový čas a paměťové prostředky.

Vytváření nových procesů probíhá pomocí dvojice systémových volání `fork()` a `exec()`. Volání `fork()` vytvoří identickou kopii rodičovského procesu, zatímco `exec()` nahradí kód běžícího procesu novým programem. Tento mechanismus je základem pro spouštění všech aplikací v systému [5].

Specifickou kategorií procesů jsou *daemony* (démoni). Jedná se o procesy, které běží na pozadí, nejsou přímo spojeny s žádným terminálem a typicky čekají na určité události nebo

poskytují systémové služby. V navržené architektuře je backend implementován právě jako daemon, který udržuje stav VPN spojení nezávisle na tom, zda je CLI klient zrovna spuštěn. Tento přístup zajišťuje persistenci spojení a umožňuje bezpečné provádění privilegovaných operací (např. manipulaci se síťovými rozhraními) pod identitou uživatele s dostatečnými právy, zatímco samotné CLI rozhraní může běžet v kontextu běžného uživatele.

Správa služeb a systémová inicializace (systemd)

Moderní linuxové distribuce využívají pro správu systému a služeb inicializační systém **systemd**. Ten v posledním desetiletí nahradil starší standardy jako **sysvinit** a přinesl zásadní změny v tom, jak jsou procesy na pozadí spouštěny, monitorovány a hierarchicky organizovány.

Zatímco starší systémy spouštěly služby sekvenčně pomocí shell skriptů, což prodlužovalo start systému, **systemd** využívá agresivní paralelizaci a spouštění služeb na základě požadavků (on-demand), například skrze socketovou aktivaci. Brian Ward vysvětluje, že **systemd** není pouhým inicializačním procesem s PID 1, ale komplexním ekosystémem pro správu uživatelského prostoru, který integruje logování (**journald**), správu zařízení (**udev**) i konfiguraci sítě [5].

Z pohledu vývoje VPN klienta jsou klíčové tzv. *Unit files* (jednotky), konkrétně **.service** soubory. Tyto deklarativní konfigurační soubory definují parametry spuštění služby, její závislosti na síťové konektivitě a bezpečnostní kontext. Díky integraci s mechanismem *cgroups* (Control Groups) má **systemd** absolutní kontrolu nad všemi procesy patřícími k dané službě, což umožňuje jejich spolehlivé ukončení nebo automatický restart v případě havárie. V navržené architektuře hraje **systemd** roli garanta běhu backendové služby, zajišťuje její spuštění po startu systému a poskytuje standardizované rozhraní pro monitoring stavu skrze nástroj **systemctl**.

Konfigurace síťového subsystému (Netlink)

Tradiční unixové nástroje ('ifconfig') byly v Linuxu nahrazeny modernějším balíkem 'iproute2'. Na pozadí těchto nástrojů stojí rozhraní **Netlink**.

Netlink slouží jako komunikační sběrnice mezi jádrem a uživatelským prostorem. Na rozdíl od 'ioctl' volání, Netlink používá standardní mechanismus socketů pro předávání zpráv o konfiguraci sítě (např. přiřazení IP adresy, změna routovací tabulky).

Meziprocesová komunikace (IPC) v Linuxu

Jelikož je aplikace rozdělena na dvě části (privilegovaná služba a neprivilegovaný klient), je nutné zajistit efektivní přenos dat mezi nimi. V prostředí Linuxu je standardem pro lokální komunikaci využití **Unix Domain Sockets (UDS)**.

Unix Domain Sockets

Na rozdíl od TCP/IP socketů, které využívají síťová rozhraní a IP adresy, UDS využívají souborový systém. Socket je reprezentován speciálním souborem na disku. Hlavní výhody pro systémové služby jsou:

1. **Výkon:** Data se nekopírují přes síťový zásobník, ale zůstávají v paměti OS.
2. **Bezpečnost a řízení přístupu:** Přístup k socketu lze omezit standardními linuxovými oprávněními souborů. To umožňuje, aby se k backendové službě mohl připojit pouze uživatel s příslušným oprávněním.

Správa oprávnění a souborový systém

Linux využívá standard **FHS (Filesystem Hierarchy Standard)**, který definuje, kam mají aplikace ukládat svá data.

Model oprávnění DAC

Linux využívá model Discretionary Access Control (DAC). Každý soubor má vlastníka (User), skupinu (Group) a sadu práv pro čtení, zápis a spuštění (**rwX**). Tento mechanismus je základem pro bezpečnost souborového systému, kde vlastník může rozhodnout, komu udělí přístup ke svým datům [5].

Pro systémové služby, jako je VPN klient, je však tradiční model DAC často nedostačující. Operace jako vytváření virtuálních síťových rozhraní (**tun/tap**), modifikace směrovacích tabulek nebo konfigurace pravidel firewallu vyžadují nejvyšší úroveň oprávnění – identitu superuživatele **root**. V prostředí Linuxu je tato moc tradičně binární: proces buď disponuje plnými právy (UID 0), nebo je omezen jako běžný uživatel.

Program **sudo** (superuser do) představuje standardní nástroj pro bezpečné delegování těchto oprávnění běžným uživatelům. V navržené architektuře běží backendová služba (daemon) právě pod identitou **root** (typicky spouštěná jako **systemd** služba s příslušnými právy), což jí umožňuje přímou manipulaci se síťovým zásobníkem jádra. Bezpečnostní bariéru zde tvoří meziprocesová komunikace: CLI frontend, běžící s právy běžného uživatele, komunikuje se službou přes Unix Domain Socket. Právě na úrovni tohoto socketu se model DAC znovu uplatňuje – přístup k souboru socketu je omezen na konkrétní uživatelskou skupinu, čímž je zaručeno, že pouze autorizovaní uživatelé mohou iniciovat VPN spojení.

Moderní Linux dále rozšiřuje tento model o tzv. *Capabilities* (schopnosti). Tento mechanismus umožňuje rozdělit absolutní moc uživatele **root** na menší, specificky zaměřené celky. Příkladem je schopnost **CAP_NET_ADMIN**, která procesu dovoluje spravovat síťová rozhraní a směrování, aniž by mu musela být udělena plná práva nad celým systémem [5]. Využití

Capabilities tak představuje implementaci principu nejmenších privilegií (Principle of Least Privilege), což výrazně zvyšuje odolnost systému proti potenciálnímu zneužití v případě kompromitace služby.

Umístění aplikace

Pro software třetích stran, který není součástí oficiální distribuce, vyhrazuje FHS adresář `/opt`. Konfigurační soubory a logy jsou standardně umísťovány do `/etc`, respektive `/var/log`.

1.4. Implementační technologie

Architektura řešení je rozdělena na privilegovanou službu (Daemon) a uživatelské rozhraní (Client).

Backend: Platforma .NET

Pro implementaci systémové služby (Daemon) byla zvolena platforma .NET 8. Tato volba vychází především z potřeby kontinuity s existující infrastrukturou GoodAccess, která je na technologiích Microsoft .NET postavena. .NET na Linuxu dnes nabízí vysoký výkon a plnou podporu pro systémové programování, včetně nativní interakce s nízkoúrovňovými API systému přes P/Invoke nebo knihovny typu Microsoft.Extensions.Hosting pro běh služeb. Modul `CLIService` tak může sdílet klíčovou logiku pro správu tokenů, certifikátů a stavu tunelu se stávajícím grafickým klientem, což zaručuje konzistentní chování aplikace napříč různými rozhraními.

Frontend: Jazyk Go a TUI

Pro klientskou část (CLI frontend) byl zvolen jazyk Go. Go se v posledních letech stal de facto standardem pro vývoj cloud-native a systémových nástrojů v ekosystému Linux (např. Docker, Kubernetes, Terraform). Hlavní výhodou je schopnost kompilace do jediné staticky linkované binárky bez nutnosti instalace runtime prostředí, což je pro nástroj příkazové řádky ideální. Go nabízí vynikající knihovny pro tvorbu moderních terminálových rozhraní – v této práci je využita knihovna `Cobra` pro definici struktury příkazů a framework `Bubble Tea` pro reaktivní zobrazení stavu a interaktivní prvky.

Meziprocesová komunikace (IPC)

V architektuře, kde je logika aplikace oddělena od uživatelského rozhraní do samostatných procesů, hraje klíčovou roli meziprocesová komunikace (IPC – Inter-Process Communication).

Pro potřeby této práce byla zvolena technologie Unix Domain Sockets (UDS).

Unix Domain Sockets představují datový komunikační koncový bod pro výměnu dat mezi procesy běžícími na stejném hostitelském operačním systému. Na rozdíl od síťových socketů (TCP/IP), které jsou určeny pro komunikaci přes síť, UDS nevyužívají síťový zásobník (network stack), což výrazně snižuje režii a zvyšuje propustnost komunikace.

Z pohledu bezpečnosti, která je u VPN klienta kritická, nabízejí UDS zásadní výhodu v podobě využití standardních souborových oprávnění systému Linux. Přístup k socketu lze omezit na konkrétního uživatele nebo skupinu, čímž se zamezí neoprávněným aplikacím v komunikaci se systémovou službou. V navržené architektuře slouží UDS jako transportní vrstva pro protokol gRPC, který zajišťuje silně typované rozhraní pro příkazy jako autentizace, připojení nebo monitoring stavu.

1.5. Distribuce softwaru

Zadání práce vyžaduje přípravu balíčků pro distribuce Debian a Fedora.

1.6. Metodika vývoje

2. Analýza a specifikace požadavků

Cílem této kapitoly je analyzovat potřeby cílové skupiny uživatelů a na jejich základě definovat funkční i nefunkční požadavky na výslednou aplikaci. Součástí je také analýza možných architektonických přístupů.

2.1. Identifikace zúčastněných stran

Primárními uživateli GoodAccess CLI pro Linux jsou technicky zdatní profesionálové:

- **System Administrators:** Spravují servery a vyžadují stabilní nástroj pro vzdálenou správu přes SSH bez nutnosti grafického rozhraní.
- **DevOps Inženýři:** Potřebují integrovat VPN připojení do automatizovaných skriptů a CI/CD pipeline. Vyžadují predikovatelné chování a strojově čitelný výstup.

2.2. Funkční požadavky

Na základě analýzy potřeb byly stanoveny následující funkční požadavky:

Autentizace a správa relace

Aplikace musí umožnit přihlášení uživatele v prostředí bez webového prohlížeče.

- **F1 – Login:** Podpora pro autentizační toky vhodné pro terminál (např. Device Code Flow), kdy uživatel potvrdí přihlášení na jiném zařízení.
- **F2 – Logout:** Bezpečné ukončení relace a smazání lokálně uložených tokenů.

Správa připojení

Jádrem aplikace je ovládání VPN tunelu.

- **F3 – Connect:** Navázání připojení k vybrané bráně. Aplikace musí podporovat výběr protokolu (WireGuard jako výchozí, OpenVPN jako záložní).
- **F4 – Disconnect:** Korektní ukončení VPN připojení a úklid routovacích pravidel.

- **F5 – Gateway Selection:** Uživatel musí mít možnost vybrat si, ke které VPN bráně (Gateway) se chce připojit.

Monitoring a stav

- **F6 – Status:** Zobrazení aktuálního stavu (Připojeno/Odpojeno, IP adresa, přenesená data).
- **F7 – JSON Output:** Možnost vyžádat si výstup příkazů (zejména status) ve formátu JSON pro snadné parsování v externích skriptech.

2.3. Nefunkční požadavky

- **N1 – Výkon:** Rychlý start aplikace (CLI). Jazyk Go byl zvolen právě pro svou rychlost spouštění a malou paměťovou náročnost.
- **N2 – Kompatibilita:** Aplikace musí být funkční na hlavních serverových distribucích Linuxu (Debian/Ubuntu, Fedora/RHEL, Arch Linux).
- **N3 – Bezpečnost:** Citlivá data (přístupové tokeny, privátní klíče) musí být uložena bezpečně a přístupná pouze oprávněné službě.

2.4. Analýza architektury

Při návrhu řešení byly zvažovány dva přístupy k interakci s existující logikou GoodAccess:

Návrh A: Tenký klient (Wrapper)

V tomto modelu běží v systému jedna privilegovaná služba (.NET), která drží stav a vykonává operace. CLI aplikace (Go) slouží pouze jako dálkové ovládání, které posílá příkazy službě přes IPC.

Návrh B: Samostatná logika

CLI aplikace by obsahovala veškerou logiku pro připojení a správu WireGuardu a běžela by přímo pod uživatelem root, nezávisle na existující službě.

Zvolené řešení

Byl zvolen **Návrh A (Wrapper)** z následujících důvodů:

1. **Single Source of Truth:** Služba drží jednotný stav aplikace. Nedochází k desynchronizaci mezi tím, co si myslí CLI, a tím, co je reálně nastaveno v síti.
2. **Bezpečnost:** Uživatel nemusí spouštět CLI pod `rootem`. Privilegované operace provádí pouze izolovaná služba na pozadí.
3. **Znovupoužitelnost:** Využívá se již otestovaná a existující logika v .NET backendu, což je v souladu s principy XP (Simplicity).

3. Návrh řešení

V této kapitole je představen detailní technický návrh aplikace. Vychází z analytických požadavků a zvolené architektury tenkého klienta.

3.1. Celková architektura systému

3.2. Návrh komunikace (IPC)

3.3. Návrh uživatelského rozhraní (CLI)

3.4. Bezpečnostní model

3.5. Datový model a konfigurace

4. Implementace

Kapitola popisuje proces vývoje, použitou strukturu projektu a klíčové implementační detaily jednotlivých komponent.

4.1. Struktura projektu a nástroje

4.2. Implementace klienta (Frontend)

4.3. Úpravy backendové služby

4.4. Integrace WireGuardu

4.5. Distribuční balíčky a Systemd

5. Testování a validace

Ověření funkčnosti a stability aplikace probíhalo v souladu s metodikou Extrémního programování průběžně během vývoje.

5.1. Metodika testování

5.2. Jednotkové testy (Unit Testing)

5.3. Integrační a systémové testy

5.4. Testování na různých distribucích

Závěr

Cílem této bakalářské práce byl návrh a implementace nativního CLI klienta GoodAccess pro OS Linux.

Shrnutí výsledků

Diskuse a přínos

Možnosti budoucího rozvoje

Seznam použitých zdrojů

1. RAYMOND, Eric Steven. *The Art of Unix Programming*. Boston: Addison-Wesley Professional, 2003. ISBN 978-0-13-142901-7.
2. GOODACCESS. *GoodAccess Documentation* [online]. 2024. [cit. 2024-11-07]. Dostupné z: <https://support.goodaccess.com/>.
3. GOODACCESS. *2. Architecture Overview* [online]. 2024. [cit. 2026-02-12]. Dostupné z: <https://support.goodaccess.com/getting-started/2.-architecture-overview>.
4. DONENFELD, Jason A. WireGuard: Next Generation Kernel Network Tunnel. In: *NDSS 2017: Network and Distributed System Security Symposium*. San Diego, 2017. Dostupné také z: <https://www.wireguard.com/papers/wireguard.pdf>.
5. WARD, Brian. *How Linux Works: What Every Superuser Should Know*. 3rd. San Francisco: No Starch Press, 2021. ISBN 978-1-7185-0040-2.

A. Externí přílohy

Externí přílohy této bakalářské práce jsou umístěny na adrese:

https://github.com/Jiri-Fiser/thesis_ki_ujep.

Na úložišti GitHub mohou být uloženy tyto externí přílohy:

- **zdrojové kódy**
- **doplňkové texty** (například jak instalovat aplikaci, manuály aplikace)
- **schémata** (především, pokud se nevejdou na stranu A4 a jejich vytištění je tak problematické)
- **screenshoty** (v textu práce lze použít jen omezený počet snímků obrazovky, které navíc nemusí být při černobílém tisku příliš přehledné)
- **videa** (například ovládání aplikace)

V každém případě by to však měli být pouze materiály, které jste vytvořili sami. Materiály jiných autorů uvádějte v seznamu použité literatury (včetně případných odkazů na jejich originální umístění).

V této kapitole stačí uvést pouze základní strukturu úložiště (co se kde nalézá a jakou má funkci) například v podobě tabulky.

ki-thesis.pdf	text práce v PDF
ki-thesis.tex	zdrojový kód práce v $\text{\LaTeX}$ u
kitheses.cls	definice třídy dokumentů (rozšířená třída <code>scrbook</code>
thesis.bib	bibliografická databáze (exportována z citace.com)
LOGO_PRF_CZ_RGB_standard.jpg	logo fakulty s českým textem
LOGO_PRF_EM_RGB_standard.jpg	logo fakulty s anglickým textem

Všechny tyto soubory jsou potřeba pro překlad dokumentu (logo stačí jedno v příslušné jazykové verzi).

B. Další přílohy

Výjimečně může práce obsahovat i další tištěné přílohy. Obecně však dávejte přednost elektronickým přílohám umístěným na GitHubu (tato kapitola tak bude úplně chybět).